

04 常用工具类

目标:把 00 总览表里散落的工具 API 全部展开Java 8 必备 API(天天用)深讲。

范围:Arrays / Collections / Math / String / StringBuilder + 包装类 Integer/Long/Double + Comparator 写法全集 + Java 8 必备。

提醒:03 容器 API 已讲过,本篇不重复讲容器,只讲"容器之外的工具"。看到容器名(ArrayList / HashMap 等)直接跳回 03。

一、开篇定调:本章定位

- 02 讲了语法基础(引用 / 字符串 / 数组 / final / static / 类与对象 / char vs String)
- 03 讲了 12 个容器对照表(ArrayList / ArrayDeque / PriorityQueue / HashMap / HashSet / LinkedList 等)
- 本篇讲"容器之外的工具":Arrays / Collections / Math / String / StringBuilder + 包装类
- 额外赠送:Comparator 5 种写法全集 + Java 8 必备 API(getOrDefault / computeIfAbsent / List.sort / String.join)

C++ 选手视角:Arrays ≈ <algorithm> 数组版,Collections ≈ <algorithm> 容器版,StringBuilder ≈ std::string 的可变性,Math ≈ <cmath>,包装类 ≈ std::to_string / std::stoi 的封装。一一对应,概念都是你会的。

二、`Arrays` (数组工具)

2.1 完整 API 速查

方法	说明	C++ 对照
<code>`static void sort(int[] a)`</code>	基础类型数组升序	<code>`std::sort`</code>
<code>`static <T> void sort(T[] a, Comparator<? super T> c)`</code>	对象数组,可自定义比较器	<code>`std::sort`</code> + lambda
<code>`static int binarySearch(int[] a, int key)`</code>	二分查找(数组必须有序)	<code>`std::binary_search`</code>
<code>`static void fill(int[] a, int val)`</code>	填充	<code>`std::fill`</code>
<code>`static String toString(int[] a)`</code>	转字符串(debug 用)	无内建
<code>`static <T> List<T> asList(T... a)`</code>	转 List(定长!)	<code>`std::vector`</code> 一次性构造
<code>`static IntStream stream(int[] a)`</code>	转 IntStream(Java 8+)	无
<code>`static int[] copyOf(int[] a, int newLength)`</code>	拷贝(可扩/缩)	<code>`std::vector`</code> 拷贝构造
<code>`static boolean equals(int[] a, int[] a2)`</code>	数组内容比较	<code>`std::equal`</code>
<code>`static void setAll(int[] a, IntUnaryOperator gen)`</code>	函数式填充(Java 8+)	<code>`std::generate`</code>

2.2 关键差异(必看)

重点:Arrays.sort(int[])(基础类型) 和 Arrays.sort(Integer[])(对象) 是两个不同重载。基础类型不能传 Comparator(编译器不让你写 (a, b) -> b - a)。

>

解决方法 1: `Arrays.sort` 配对象类型: `Arrays.sort(Integer[], (a, b) -> b - a)`。

解决方法 2: 转成 `List` 排序, 见下。

2.3 最小代码: 排序 + 自定义比较器 + 二分 + 填充

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 基础类型升序
        int[] a = {3, 1, 4, 1, 5, 9, 2, 6};
        Arrays.sort(a);
        System.out.println(Arrays.toString(a)); // [1, 1, 2, 3, 4, 5, 6, 9]

        // 2. 基础类型降序? 不能直接 sort! 两种解法
        // 解法 A: 包装成 Integer[] 再 sort + Comparator
        Integer[] a2 = {3, 1, 4, 1, 5, 9, 2, 6};
        Arrays.sort(a2, (x, y) -> y - x); // 降序
        System.out.println(Arrays.toString(a2)); // [9, 6, 5, 4, 3, 2, 1, 1]

        // 解法 B: 转 List 用 Collections.sort(见下节)

        // 3. 二维数组按第二个字段升序(LeetCode 56 合并区间简化)
        int[][] points = {{1, 3}, {2, 2}, {3, 1}};
        Arrays.sort(points, (p, q) -> p[1] - q[1]); // 按 [1] 升序
        // 输出: [[3, 1], [2, 2], [1, 3]]

        // 4. 二分查找(必须先排序!)
        int[] sorted = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        int idx = Arrays.binarySearch(sorted, 5); // 返回 4(index)
        int miss = Arrays.binarySearch(sorted, 10); // 返回 -10(负数表示未找到)

        // 5. 填充
        int[] fillArr = new int[5];
        Arrays.fill(fillArr, 7);
        System.out.println(Arrays.toString(fillArr)); // [7, 7, 7, 7, 7]

        // 6. 数组比较
        int[] x = {1, 2, 3}, y = {1, 2, 3};
        System.out.println(Arrays.equals(x, y)); // true(不是 == !)
    }
}
```

2.4 C++ 对照



```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> a = {3, 1, 4, 1, 5, 9, 2, 6};
    sort(a.begin(), a.end());          // 升序
    sort(a.rbegin(), a.rend());        // 降序(反迭代器)
    sort(a.begin(), a.end(), greater<int>()); // 降序(标准写法)

    // 二分
    bool found = binary_search(a.begin(), a.end(), 5);

    // 填充
    vector<int> fillArr(5);
    fill(fillArr.begin(), fillArr.end(), 7);
    return 0;
}

```

2.5 坑点: `Arrays.asList` 返回定长 list(详见 06 坑 5)

```

// 错误:UnsupportedOperationException
List<Integer> list = Arrays.asList(1, 2, 3);
list.add(4);

// 正确 1:外面包一层 ArrayList
List<Integer> mutable = new ArrayList<>(Arrays.asList(1, 2, 3));
mutable.add(4); // OK

// 正确 2:用 List.of(java 9+,刷题可能用不上)
List<Integer> immutable = List.of(1, 2, 3);

```

03 容器篇第 188-199 行有详细讲解,这里记住一句话: `Arrays.asList(...)` 当参数传可以,不要拿去 add/re move。

三、`Collections` (集合工具)

3.1 完整 API 速查

方法	说明	C++ 对照
`static <T> void sort(List<T> list)`	列表升序(元素必须实现 Comparable)	`std::sort`
`static <T> void sort(List<T> list, Comparator<? super T> c)`	自定义比较器排序	`std::sort` + lambda
`static void reverse(List<?> list)`	原地反转	`std::reverse`
`static T max(Collection<? extends T> coll)`	最大值	`std::max_element`



<code>`static T min(Collection<? extends T> coll)`</code>	最小值	<code>`std::min_element`</code>
<code>`static int frequency(Collection<?> c, Object o)`</code>	出现次数	<code>`std::count`</code>
<code>`static void swap(List<?> list, int i, int j)`</code>	交换两个位置	<code>`std::swap`</code>
<code>`static int binarySearch(List<? extends Comparable<? super T>> list, T key)`</code>	二分(列表必须有序)	<code>`std::binary_search`</code>
<code>`static boolean addAll(Collection<? super T> c, T... elements)`</code>	批量添加	<code>`std::vector::insert`</code>
<code>`static boolean disjoint(Collection<?> c1, Collection<?> c2)`</code>	是否不相交(无共同元素)	无内建,需手写

3.2 最小代码:list 排序 + 反转 + 最值

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6));

        // 1. 排序
        Collections.sort(list);           // 升序:[1, 1, 2, 3, 4, 5, 6, 9]
        Collections.sort(list, (a, b) -> b - a); // 降序:[9, 6, 5, 4, 3, 2, 1, 1]

        // 2. 反转
        Collections.reverse(list); // 翻转列表

        // 3. 最大 / 最小(返回元素,不是下标)
        int maxVal = Collections.max(list); // 9
        int minVal = Collections.min(list); // 1

        // 4. 频次
        int cnt = Collections.frequency(list, 1); // 1 出现几次

        // 5. 交换
        Collections.swap(list, 0, 1); // 交换第 0 和第 1 个

        // 6. 批量添加
        List<Integer> list2 = new ArrayList<>();
        Collections.addAll(list2, 1, 2, 3, 4, 5); // 等价于 add 5 次

        // 7. 不相交判断
        boolean noCommon = Collections.disjoint(
            Arrays.asList(1, 2, 3),
            Arrays.asList(4, 5, 6)
        ); // true
    }
}
```

3.3 C++ 对照



```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> list = {3, 1, 4, 1, 5, 9, 2, 6};

    sort(list.begin(), list.end());          // 升序
    sort(list.rbegin(), list.rend());        // 降序
    reverse(list.begin(), list.end());       // 反转
    int maxVal = *max_element(list.begin(), list.end()); // 9(注意要解引用)
    int minVal = *min_element(list.begin(), list.end()); // 1
    int cnt = count(list.begin(), list.end(), 1); // 频次
    swap(list[0], list[1]);                  // 交换

    return 0;
}

```

3.4 关键差异(Java 8 后用 `List.sort` 优先)

重要:Collections.sort(list) 和 list.sort(comparator) 功能等价。但 List.sort 是 Java 8 新增的 List 接口默认方法,写法更简洁、稍快一点。Collections.sort 是 Java 8 之前的"老写法",现已不推荐。

>

C++ 选手注意:C++ 是 sort(v.begin(), v.end())(算法 + 迭代器),Java 是 list.sort(c)(方法是 List 自己的)。调用对象不同,效果一样。

写法	推荐度	备注
`list.sort((a, b) -> a - b)`	★★★	Java 8+,默认写法
`Collections.sort(list)`	★★	老写法,仅出现在早期题解
`Collections.sort(list, (a, b) -> a - b)`	★★	老写法 + 自定义

详见本章第九节 List.sort。

四、`Math` (数学工具)

4.1 完整 API 速查

方法	说明	C++ 对照
`static int max(int a, int b)` / `long` / `double`	两数取大	`std::max`
`static int min(int a, int b)` / `long` / `double`	两数取小	`std::min`
`static int abs(int a)` / `long` / `double`	绝对值	`std::abs`
`static double pow(double a, double b)`	a 的 b 次方	`std::pow`
`static double sqrt(double a)`	平方根	`std::sqrt`
`static double log(double a)`	自然对数(底 e)	`std::log`
`static double log10(double a)`	常用对数(底 10)	`std::log10`



`static double sin/cos/tan(double a)`	三角函数(参数是弧度)	`std::sin/cos/tan`
`static double PI`	3.14159...	`M_PI`
`static double E`	2.71828...	`M_E`
`static double random()`	[0, 1) 随机 double	`std::rand` / ` <random>`</random>

4.2 最小代码:距离公式 + 浮点比较

```
public class Main {
    public static void main(String[] args) {
        // 1. 基础运算
        int a = Math.max(3, 5);    // 5
        int b = Math.abs(-7);     // 7
        double c = Math.sqrt(16);  // 4.0
        double d = Math.pow(2, 10); // 1024.0
        double pi = Math.PI;      // 3.14159...

        // 2. 三角函数(弧度!)
        double rad = Math.toRadians(30); // 30 度 → 弧度
        double s = Math.sin(rad);        // 0.5

        // 3. 随机数
        double r = Math.random(); // [0, 1)
        int dice = (int) (Math.random() * 6) + 1; // [1, 6]

        // 4. 浮点比较(★ 浮点精度坑)
        double x = 0.1 + 0.2;
        System.out.println(x);          // 0.30000000000000004(不是 0.3!)
        System.out.println(x == 0.3);    // false(直接 == 永远 false)
        System.out.println(Math.abs(x - 0.3) < 1e-9); // true(eps 比较)

        // 5. 实际应用:两点欧氏距离
        double dist = Math.sqrt(Math.pow(3 - 0, 2) + Math.pow(4 - 0, 2)); // 5.0
    }
}
```

4.3 C++ 对照



```
#include <cmath>
#include <iostream>
using namespace std;

int main() {
    int a = max(3, 5);
    int b = abs(-7);
    double c = sqrt(16);
    double d = pow(2, 10);
    double pi = M_PI;

    // 浮点比较
    double x = 0.1 + 0.2;
    if (abs(x - 0.3) < 1e-9) { /* 相等 */ }

    return 0;
}
```

4.4 坑点:浮点精度,不要直接 `==` (详见 06 坑 7)

```
// 错误:浮点数永远不要直接 ==
double a = 0.1 + 0.2;
if (a == 0.3) { /* 永远不会进 */ }

// 正确:eps 比较
final double EPS = 1e-9;
if (Math.abs(a - 0.3) < EPS) { /* 相等 */ }
```

原因:IEEE 754 浮点数无法精确表示 0.1、0.2、0.3,所有浮点运算都有微小误差。所有浮点比较都要加 eps(epsilon,1e-9 是常用阈值)。

五、`String` (字符串,不可变)

Java 不可变:String 一旦创建就不能改。s = s + "x" 实际上是创建了一个新 String 对象,把引用赋给 s。要"原地改"用 StringBuilder(见下节)。

>

C++ 选手视角:C++ 的 std::string 是可变的,s += "x" 是真改原对象。Java 的 String 概念上更接近 const std::string,或者 char* p = "abc"(只读指针)。这是 C++ 转 Java 第一个概念落差,务必记牢。

5.1 完整 API 速查

方法	说明	C++ 对照
`char charAt(int index)`	取 index 字符	`s[i]`
`int indexOf(String str)` / `lastIndexOf`	找子串位置(返回 -1 表示没有)	`s.find(...)`
`String substring(int begin)` / `substring(int begin, int end)`	子串(包头不包尾)	`s.substr(...)`
`boolean equals(Object anObject)`	内容比较	`s == t` (C++ 内容比较)
`int hashCode()`	哈希值	无



<code>`boolean contains(CharSequence s)`</code>	是否包含子串	<code>`s.find(...) != string::npos`</code>
<code>`boolean startsWith(String p)` / `endsWith(String s)`</code>	前/后缀	<code>`s.rfind(p) == 0`</code>
<code>`int length()`</code>	长度(有括号!)	<code>`s.length()`</code> (C++ 也有括号)
<code>`String toLowerCase()`` / `toUpperCase()`</code>	大小写转换	<code>`std::transform`</code>
<code>`String trim()`</code>	去首尾空白(不删中间)	无内建,需手写
<code>`String replace(old, new)`` / `replaceAll(regex, new)`</code>	替换(后者正则)	<code>`s.replace(...)`</code>
<code>`String[] split(String regex)`</code>	切分(参数是正则)	<code>`std::stringstream`</code>
<code>`String concat(String str)`</code>	拼接(返回新串)	<code>`s + str`</code>
<code>`int compareTo(String s)`</code>	字典序比较	<code>`<` / `>`</code>

5.2 最小代码:分割 + 查找 + 替换

```
public class Main {
    public static void main(String[] args) {
        String s = "Hello, World! Hello, Java!";

        // 1. 分割(注意:参数是正则!)
        String[] parts = s.split(", "); // ["Hello", "World! Hello", "Java!"]
        // 切"."这种元字符要转义
        String[] words = "a.b.c".split("\\."); // ["a", "b", "c"]

        // 2. 查找
        int idx = s.indexOf("World"); // 7(返回位置)
        int noExist = s.indexOf("xxx"); // -1(没找到)

        // 3. 子串
        String sub = s.substring(7, 12); // "World"(包头不包尾)

        // 4. 替换
        String r1 = s.replace("Hello", "Hi"); // 替换所有
        String r2 = s.replaceAll("\\s+", "_"); // 正则:把空白换成 _

        // 5. 大小写 + 去空白
        String lower = "ABC".toLowerCase(); // "abc"
        String trimmed = " abc ".trim(); // "abc"

        // 6. 是否包含
        boolean has = s.contains("Java"); // true
    }
}
```

5.3 C++ 对照



```

#include <string>
#include <iostream>
#include <sstream>
using namespace std;

int main() {
    string s = "Hello, World! Hello, Java!";

    // 分割(用 stringstream,比 Java 麻烦)
    stringstream ss(s);
    string item;
    while (getline(ss, item, ',')) { /* 处理 item */ }

    // 查找
    int idx = s.find("World");    // 返回位置
    int noExist = s.find("xxx");  // string::npos(没找到)

    // 子串
    string sub = s.substr(7, 5);   // "World"(长度 5)

    // 替换
    // C++ 的 replace 比较绕,通常用循环或正则库

    return 0;
}

```

5.4 坑点 1: `==` vs `equals` (C++ 选手第 1 大坑,详见 06 坑 11)

C++ 选手的致命错:在 C++ 里 `s == t` 是内容比较。Java 里 `==` 是引用比较(比地址),`equals` 才是内容比较。C++ 选手转 Java 几乎必踩。

```

// 错误:== 比的是引用,不是内容
String s1 = "abc";
String s2 = new String("abc");
System.out.println(s1 == s2);    // false!(比的是地址)
System.out.println(s1.equals(s2)); // true(内容比较)

// 更隐蔽的坑:字符串常量池
String a = "hello";
String b = "hello";
System.out.println(a == b);      // true(都指向常量池同一对象)
String c = new String("hello");
System.out.println(a == c);      // false(a 是常量池,c 是 new 出来的)

```

```

// C++ 没这个问题,s == t 直接比内容
string s1 = "abc";
string s2 = "abc";
cout << (s1 == s2); // true,无误

```

金句:"在 Java 里,字符串比内容永远用 `equals()`,`==` 只用于比 `null` 或比对象地址"。这条规则适用于 `String` / `Integer` / `Long` / `Double` / 任何包装类。



5.5 坑点 2: `length` vs `length()` vs `size()` (详见 06 坑 12)

C++ 选手困惑: C++ 全靠 `s.size()` / `v.size()`。Java 分三套: 数组用 `length` (无括号), 字符串用 `length()` (有括号), 集合用 `size()` (有括号)。第一次写会卡。

```
int[] arr = {1, 2, 3};
int a = arr.length;      // 数组: 字段, 无括号
int b = "abc".length();   // 字符串: 方法, 有括号
int c = list.size();      // List/Set/Map: 方法, 有括号
int d = map.size();       // Map 也有 size()
```

记忆口诀: 数组 `length` 字段不挂括号, 字符串 `length` 挂括号, 集合 `size` 挂括号。C++ 全是 `.size()` 一统天下, Java 因为历史原因分成 3 套。

六、`StringBuilder` (可变字符串, 刷题天天用)

概念铺垫: 因为 `String` 不可变, 所有“原地改字符串”的需求(拼接、删除、反转)都用 `StringBuilder`。刷题时 80% 的“我要拼一个长字符串”场景, 都用 `StringBuilder`。

6.1 完整 API 速查

方法	说明	C++ 对照
<code>StringBuilder()</code>	构造空(默认容量 16)	<code>string s;</code>
<code>StringBuilder(int capacity)</code>	构造指定容量	<code>string s; s.reserve(n);</code>
<code>StringBuilder(String str)</code>	构造初始字符串	<code>string s = "abc";</code>
<code>StringBuilder append(...)</code>	拼接(任意类型, 返回 <code>this</code> , 可链式)	<code>s += x;</code>
<code>StringBuilder reverse()</code>	原地反转	<code>std::reverse(s.begin(), s.end())</code>
<code>String toString()</code>	转 <code>String</code>	<code>s.c_str()</code> / <code>static_cast<string>(s)</code>
<code>StringBuilder deleteCharAt(int index)</code>	删 <code>index</code> 字符	<code>s.erase(s.begin() + i)</code>
<code>StringBuilder setCharAt(int index, char ch)</code>	改 <code>index</code> 字符	<code>s[i] = ch;</code>
<code>StringBuilder insert(int offset, ...)</code>	在 <code>offset</code> 插入	<code>s.insert(i, x)</code>
<code>int length()</code>	当前长度	<code>s.length()</code>
<code>char charAt(int index)</code>	读 <code>index</code> 字符	<code>s[i]</code>

6.2 最小代码: 数字反转(LeetCode 7) + 字符串拼接



```

public class Main {
    public static void main(String[] args) {
        // 1. 数字反转(LeetCode 7 整数反转 简化版)
        int x = 12345;
        StringBuilder sb = new StringBuilder(String.valueOf(x)); // "12345"
        sb.reverse(); // "54321"
        int result = Integer.parseInt(sb.toString()); // 54321

        // 2. 字符串拼接(★ 刷题天天用)
        // 错误:N 个字符串 + 拼接 = O(n²) 慢!
        String s = "";
        for (int i = 0; i < 100000; i++) {
            s += "x"; // 每次都创建新 String 对象,慢
        }
        // 正确:StringBuilder 拼接 = O(n) 快
        StringBuilder fast = new StringBuilder();
        for (int i = 0; i < 100000; i++) {
            fast.append("x");
        }
        String out = fast.toString();

        // 3. 链式调用
        String s2 = new StringBuilder()
            .append("Hello")
            .append(", ")
            .append("World")
            .reverse() // "dlroW ,olleH"
            .toString();

        // 4. 改单个字符
        StringBuilder sb2 = new StringBuilder("abc");
        sb2.setCharAt(0, 'z'); // "zbc"
        sb2.deleteCharAt(1); // "zc"
    }
}

```

6.3 C++ 对照



```

#include <string>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    // 1. 数字反转
    string s = to_string(12345); // "12345"
    reverse(s.begin(), s.end()); // "54321"
    int result = stoi(s); // 54321

    // 2. 字符串拼接
    string out;
    for (int i = 0; i < 100000; i++) out += "x"; // C++ string 可变,直接 += 就好
    // C++ 不需要 StringBuilder 因为 std::string 本来就可变!

    return 0;
}

```

6.4 关键提醒:为什么需要 `StringBuilder`?

C++ 选手注意:C++ 的 `std::string` 本身就是可变的,直接 `s += "x"` 就行。Java 的 `String` 不可变(像 `const string`),所以才有 `StringBuilder`(像 C++ 的 `std::string`)。

>

刷题金句:"所有要拼接 N 个字符串的场景,都先用 `StringBuilder`"。如果你看到代码里用 `String s = ""; for (... ) s += x;` 这种写法,99% 是错的(LeetCode 慢、超大数据 TLE)。

6.5 实战示例:LeetCode 43 字符串相乘(简化版)

```

// 思路:大数乘法用 StringBuilder 倒着累加,避免 BigInteger 的开销
public String multiply(String num1, String num2) {
    if (num1.equals("0") || num2.equals("0")) return "0";
    int m = num1.length(), n = num2.length();
    int[] pos = new int[m + n]; // 每一位的累加和
    for (int i = m - 1; i >= 0; i--) {
        for (int j = n - 1; j >= 0; j--) {
            int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
            int p1 = i + j, p2 = i + j + 1;
            int sum = mul + pos[p2];
            pos[p1] += sum / 10;
            pos[p2] = sum % 10;
        }
    }
    StringBuilder sb = new StringBuilder();
    for (int p : pos) {
        if (!(sb.length() == 0 && p == 0)) sb.append(p);
    }
    return sb.length() == 0 ? "0" : sb.toString();
}

```



七、包装类 `Integer` / `Long` / `Double`

7.1 完整 API 速查(以 `Integer` 为例, `Long` / `Double` 同名同义)

方法	说明	C++ 对照
<code>`static int parseInt(String s)`</code>	字符串转 int(基础类型)	<code>`std::stoi(s)`</code>
<code>`static Integer valueOf(String s)`</code>	字符串转 Integer(对象)	无(直接 `int`)
<code>`static String toString(int i)`</code>	int 转字符串	<code>`std::to_string(i)`</code>
<code>`static int MAX_VALUE`</code> / <code>`MIN_VALUE`</code>	极值($2^{31}-1$ / -2^{31})	<code>`INT_MAX`</code> / <code>`INT_MIN`</code>
<code>`static int bitCount(int i)`</code>	二进制 1 的个数	<code>`__builtin_popcount(i)`</code>
<code>`static int signum(int i)`</code>	符号(1/0/-1)	无内建
<code>`static String toBinaryString(int i)`</code>	二进制字符串	<code>`std::bitset`</code>
<code>`static int compare(int x, int y)`</code>	比较	<code>`<`</code> / <code>`>`</code>
<code>`static int parseUnsignedInt(String s)`</code>	无符号解析	无

同系列: `Long.parseLong` / `Long.bitCount` / `Double.parseDouble` / `Double.MAX_VALUE` 等。

7.2 最小代码: `parseInt` / `bitCount` / 大数判断



```

public class Main {
    public static void main(String[] args) {
        // 1. 字符串 ↔ 数字
        int x = Integer.parseInt("123");    // 123
        String s = Integer.toString(123);    // "123"

        // 2. 极值(防溢出)
        int INF = Integer.MAX_VALUE;        //  $2^{31} - 1 = 2147483647$ 
        long BIG = Long.MAX_VALUE;          //  $2^{63} - 1$ 

        // 3. 二进制 1 的个数(LeetCode 191/338 天天用)
        int cnt = Integer.bitCount(7);       // 7 = 1112 ,返回 3
        int cnt2 = Integer.bitCount(8);       // 8 = 10002 ,返回 1

        // 4. ★ 数字反转后判断是否溢出(LeetCode 7, LeetCode 模式可提交版见 class Solution)
        int reversed = new Solution().reverse(123); // 调用 Solution.reverse, 返回 321

        // 5. Long 装大数(LeetCode 2 两数相加)
        long big = Long.parseLong("9223372036854775807"); // Long.MAX_VALUE
    }
}

// LeetCode 模式可提交版
class Solution {
    public int reverse(int x) {
        int res = 0;
        while (x != 0) {
            int pop = x % 10;
            x /= 10;
            // 溢出判断
            if (res > Integer.MAX_VALUE / 10) return 0;
            if (res < Integer.MIN_VALUE / 10) return 0;
            res = res * 10 + pop;
        }
        return res;
    }
}

```

7.3 C++ 对照

```

#include <string>
#include <climits>
#include <iostream>
using namespace std;

int main() {
    int x = stoi("123");    // 字符串转 int
    string s = to_string(123); // int 转字符串
    int INF = INT_MAX;      // 极值
    int cnt = __builtin_popcount(7); // 二进制 1 的个数 → 3
    return 0;
}

```



7.4 坑点: `Integer` 缓存池(参考 03 装箱小框,详见 06 坑 3)

核心陷阱: `Integer a = 100; Integer b = 100; a == b` 是 `true`(在 `-128~127` 范围内,自动装箱用缓存池,指向同一对象)。但 `a = 200; b = 200; a == b` 是 `false`(超出缓存池,`new Integer(200)` 创建不同对象)。

```
Integer a = 100, b = 100;
System.out.println(a == b); // true(缓存池范围 -128~127)
System.out.println(a.equals(b)); // true(永远用 equals)

Integer c = 200, d = 200;
System.out.println(c == d); // false!超出缓存池
System.out.println(c.equals(d)); // true(永远用 equals)
```

```
// C++ 没这个问题,int 是基本类型,== 就是值比较
int a = 100, b = 100;
cout << (a == b); // true,无误
```

金句:“`Integer` / `Long` / `Double` / `String` 比较,永远 `.equals()` ,永远 `.equals()` ,永远 `.equals()` ”。说三遍。03 装箱小框(第 49-73 行) 有完整铺垫,本节只是复习。

7.5 实战金句

- `Integer` 与 `int` 自动互转:`int i = 5; Integer a = i;`(装箱)/ `int j = a;`(拆箱)。Java 编译器自动做。
- `Integer` 运算自动拆箱:`Integer a = 5; a++` 实际上先拆箱成 `int`,加 1,再装箱回 `Integer`。装箱拆箱是免费的,不用担心性能。
- `Math.abs(Integer.MIN_VALUE)` 是负数:`Integer.MIN_VALUE = -2^31`,绝对值是 `2^31` 超出 `int` 范围。要用 `Math.abs(long x)`。

八、`Comparator` 写法全集(5 种,02 迁入)

本章定位: `Comparator` 是 Java 排序的灵魂。本节列所有写法,05 排序模板里再讲“怎么用”。背下这 5 种,90% 排序场景够用。

8.1 写法 1:实现接口(老派,实际不写)

```
import java.util.*;

class MyComparator implements Comparator<Integer> {
    @Override
    public int compare(Integer a, Integer b) {
        return b - a; // 降序
    }
}

public class Main {
    public static void main(String[] args) {
        Integer[] arr = {3, 1, 4, 1, 5, 9, 2, 6};
        Arrays.sort(arr, new MyComparator()); // 传实例
    }
}
```



适用场景:比较器逻辑复杂且多处复用。实际刷题几乎不用,知道有这写法就行。

8.2 写法 2:匿名内部类(Java 7 时代主流)

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Integer[] arr = {3, 1, 4, 1, 5, 9, 2, 6};
        Arrays.sort(arr, new Comparator<Integer>() {
            @Override
            public int compare(Integer a, Integer b) {
                return b - a; // 降序
            }
        });
    }
}
```

适用场景:Java 7 及之前写法,现在已被 lambda 取代。刷题不用,看到题解能读懂就行。

8.3 写法 3:lambda(Java 8+, 刷题主力)

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Integer[] arr = {3, 1, 4, 1, 5, 9, 2, 6};

        // 升序
        Arrays.sort(arr, (a, b) -> a - b);

        // 降序
        Arrays.sort(arr, (a, b) -> b - a);

        // PriorityQueue 大顶堆
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);

        // 二维数组按第二个字段升序
        int[][] points = {{1, 3}, {2, 2}, {3, 1}};
        Arrays.sort(points, (a, b) -> a[1] - b[1]);
    }
}
```

适用场景:90% 刷题场景都用这个。

8.4 写法 4:链式调用(多键排序神器)

第一次见的概念铺垫:对比单字段排序((a, b) -> a[0] - b[0]),链式调用是多键排序——先按一个字段排,字段相同再按下一个字段排。LeetCode 406 / 179 / 56 等多键排序题必备。



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 多键排序:按第二个字段升序,再按第一个字段升序
        int[][] points = {{1, 3}, {2, 2}, {3, 1}, {1, 2}};
        Arrays.sort(points, Comparator
            .comparingInt((int[] p) -> p[1]) // 主键:第二个字段
            .thenComparingInt(p -> p[0])); // 次键:第一个字段
        // 排完:[1, 2], [3, 1], [2, 2], [1, 3]

        // 2. 字符串 + 数字混合排序
        String[] words = {"apple", "banana", "cherry"};
        Arrays.sort(words, Comparator
            .comparingInt(String::length) // 按长度升序
            .thenComparing(String::compareTo)); // 长度相同按字典序

        // 3. 降序链式
        Arrays.sort(points, Comparator
            .comparingInt((int[] p) -> p[1])
            .reversed()); // 整体反转 -> 第二个字段降序

        // 4. 链式 + 然后升序(降序降序)
        // LeetCode 406 标准:先 h 降序,再 k 升序
        int[][] people = {{7, 0}, {4, 4}, {7, 1}, {5, 0}, {6, 1}, {5, 2}};
        Arrays.sort(people, Comparator
            .comparingInt((int[] p) -> p[0]).reversed() // h 降序
            .thenComparingInt(p -> p[1])); // k 升序
    }
}

```

关键 API:

- `Comparator.comparingInt(ToIntFunction)` - 按 int 字段升序
- `Comparator.comparingLong(ToLongFunction)` - 按 long 字段升序
- `Comparator.comparing(Function)` - 按对象字段升序
- `.thenComparingInt(...)` - 第二键(int)
- `.thenComparingLong(...)` - 第二键(long)
- `.thenComparing(...)` - 第二键(对象)
- `.reversed()` - 整体反转(降序)

适用场景:多键排序必用,比手写 lambda $((a, b) \rightarrow a[0] \neq b[0] ? a[0] - b[0] : a[1] - b[1])$ 简洁 5 倍。

8.5 写法 5: `Collections.reverseOrder()` (最简降序)



```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Integer[] arr = {3, 1, 4, 1, 5, 9, 2, 6};
        Arrays.sort(arr, Collections.reverseOrder()); // 降序,一行搞定

        // 也可以用 Comparator.reverseOrder()(Java 8+)
        Arrays.sort(arr, Comparator.reverseOrder());
    }
}
```

适用场景:单字段降序用这个最简洁,比 $(a, b) \rightarrow b - a$ 还能防止整数溢出(虽然一般不会)。

8.6 记忆口诀(必背)

场景	写法	备注
升序(单字段)	<code>(a, b) -> a - b</code>	最常用
降序(单字段)	<code>(a, b) -> b - a</code> 或 <code>Collections.reverseOrder()</code>	后者更安全
升序(多字段)	<code>Comparator.comparingInt(...).thenComparingInt(...)</code>	多键神器
降序(多字段)	<code>.comparingInt(...).reversed().thenComparingInt(...)</code>	整体反转
字符串字段	<code>Comparator.comparing(String::length)</code>	方法引用

实战金句:"单字段 lambda,多字段链式,降序优先 reverseOrder"。

九、Java 8 必备 API(深讲,刷题天天用)

本章定位:这 4-5 个 API 是 Java 8 之后刷题最常用的,直接决定代码是不是"现代写法"。LeetCode 1/4 9/387/606 等题标准答案全用这些。

9.1 `Map.getOrDefault(key, defaultValue)`(LeetCode 1/347 天天用)

C++ 选手视角:C++ 用 `m.count(k) ? m[k] : default;` 或 `m.find(k) != m.end() ? m[k] : default;`(2-3 行)。Java 一行搞定,这就是 Java 8 的甜头。

场景:统计字符/元素出现次数,key 不存在时给个默认值。



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 统计频次(最常用)
        Map<Character, Integer> freq = new HashMap<>();
        String s = "abracadabra";
        for (char c : s.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }
        System.out.println(freq); // {a=5, b=2, r=2, c=1, d=1}

        // 2. LeetCode 387 字符串中第一个唯一字符(完整版见 class Solution)
        // 思路:统计每个字符出现次数,再扫一遍找第一个 count == 1 的
        int idx = new Solution().firstUniqChar("abracadabra"); // 调用,返回 4('c' 是第一个唯一字符)
        System.out.println(idx);

        // 3. C++ 对照
        // unordered_map<char, int> freq;
        // for (char c : s) freq[c]++;
        // // C++ 的 unordered_map operator[] 会自动默认初始化为 0,不需要 getOrDefault
    }
}

// LeetCode 模式可提交版
class Solution {
    public int firstUniqChar(String s) {
        Map<Character, Integer> count = new HashMap<>();
        for (char c : s.toCharArray()) {
            count.put(c, count.getOrDefault(c, 0) + 1);
        }
        for (int i = 0; i < s.length(); i++) {
            if (count.get(s.charAt(i)) == 1) return i;
        }
        return -1;
    }
}

```

金句:"统计频次 = 循环 + getOrDefault + 1",是 LeetCode 哈希题第一模板。

9.2 `Map.computeIfAbsent(key, k -> new ArrayList<>())` (BFS/图遍历建邻接表)

C++ 选手视角:C++ 没有 computeIfAbsent,要 find + insert 写 3-4 行。Java 一行搞定,直接起飞。

场景:邻接表(BFS/DFS)、按 key 分组。



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 建邻接表(LeetCode 49 字母异位词分组 简化版)
        String[] strs = {"eat", "tea", "tan", "ate", "nat", "bat"};
        Map<String, List<String>> groups = new HashMap<>();
        for (String s : strs) {
            char[] ca = s.toCharArray();
            Arrays.sort(ca);
            String key = new String(ca);
            // ★ 一行建邻接表:无 key 就 new ArrayList,再 add
            groups.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        System.out.println(new ArrayList<>(groups.values()));
        // 输出:[[eat, tea, ate], [tan, nat], [bat]]

        // 2. 图的邻接表(LeetCode 课程表 / 单词接龙 等)
        // 假设有边:1->2, 1->3, 2->3
        int[][] edges = {{1, 2}, {1, 3}, {2, 3}};
        Map<Integer, List<Integer>> graph = new HashMap<>();
        for (int[] e : edges) {
            graph.computeIfAbsent(e[0], k -> new ArrayList<>()).add(e[1]);
        }
        System.out.println(graph); // {1=[2, 3], 2=[3]}

        // 3. C++ 对照(需要 3-4 行)
        // auto it = groups.find(key);
        // if (it == groups.end()) groups[key] = vector<string>{};
        // groups[key].push_back(s);
    }
}

```

金句:“`computeIfAbsent` = 没有就建,有就返回”,比 `getOrDefault` + `put` 简洁一倍,是 Java 8 写邻接表的标配。

>

03 容器篇第 631-656 行有更详细的解释,这里记住一行模板就行。

9.3 `List.sort(Comparator)` (Java 8 新增,`Collections.sort` 替代)

C++ 选手视角:C++ 是 `std::sort(v.begin(), v.end())` (算法 + 迭代器)。Java 是 `list.sort(c)` (方法是 List 自己的)。调用对象不同,效果一样。

关键差异:

维度	`Collections.sort(list)`	`List.sort(comparator)`
引入版本	Java 2	Java 8
调用方式	工具类静态方法	List 接口默认方法
可读性	老式	新式,推荐
性能	略慢	略快(内部走 `Arrays.sort`)



```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6));

        // 写法 1: List.sort(推荐 Java 8+)
        list.sort((a, b) -> a - b);    // 升序
        list.sort(Comparator.reverseOrder()); // 降序

        // 写法 2: Collections.sort(老式 Java 8 之前)
        Collections.sort(list);        // 升序
        Collections.sort(list, (a, b) -> b - a); // 降序

        // 两种写法功能等价, 优先用 List.sort
    }
}
```

金句: "刷题默认用 `List.sort`, 看到 `Collections.sort` 知道是老写法"。

9.4 `String.join(separator, elements)` (LeetCode 打印列表神器)

C++ 选手视角: C++ 没内建 join, 得用 `std::accumulate + lambda` 写 3-4 行。Java 一行搞定。

场景: 把 `List<String>` / 字符串数组用分隔符拼成字符串。LeetCode 打印路径、构造格式化输出必备。



```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        // 1. 拼 List<String>
        List<String> list = Arrays.asList("1", "2", "3");
        String s1 = String.join("-> ", list); // "1 -> 2 -> 3"

        // 2. 拼字符串数组(变体)
        String s2 = String.join(", ", "a", "b", "c"); // "a, b, c"
        String s3 = String.join("-", new String[]{"x", "y", "z"}); // "x-y-z"

        // 3. LeetCode 606 根据二叉树创建字符串(完整版见 class Solution)
        // 思路:前序遍历 + 拼接括号
        // 假设 root 是已构造好的二叉树
        // String treeStr = new Solution().tree2str(root);

        // 4. LeetCode 416 分割等和子集:打印路径
        List<Integer> path = Arrays.asList(1, 2, 3, 4);
        String pathStr = String.join("->", path.stream().map(String::valueOf).collect(Collectors.toList()));
        // 路径: "1->2->3->4"

        // 5. C++ 对照(无内建 join)
        // string s = "";
        // for (int i = 0; i < list.size(); i++) {
        //     if (i > 0) s += "-> ";
        //     s += list[i];
        // }
    }
}

// LeetCode 606 可提交版(配合 LeetCode 606 题目的 TreeNode 节点类)
class Solution {
    public String tree2str(TreeNode root) {
        if (root == null) return "";
        String s = String.valueOf(root.val);
        if (root.left == null && root.right == null) return s;
        s += "(" + tree2str(root.left) + ")";
        if (root.right != null) {
            s += "(" + tree2str(root.right) + ")";
        }
        return s;
    }
}

```

金句:"打印路径 / 列表 → `String.join(分隔符, list)`",免去手写循环。

9.5 `String.chars()` / `String.codePoints()` (扫一眼即可)

用途:字符流式处理,Java 8+ 才有。速成不深讲,实际刷题用 `toCharArray()` 代替。



```

public class Main {
    public static void main(String[] args) {
        String s = "abc";

        // 1. chars():返回 IntStream(每个元素的 ASCII/Unicode 码点)
        s.chars().forEach(c -> System.out.println((char) c));
        // 输出:a b c(各一行)

        // 2. codePoints():返回完整 Unicode 码点 IntStream(处理中文/emoji 更准)
        s.codePoints().forEach(c -> System.out.println((char) c));

        // 3. 推荐替代:toArray() 更直观
        for (char c : s.toCharArray()) {
            System.out.println(c);
        }
    }
}

```

金句:"看到 `chars()` / `codePoints()` 知道是流式处理,实际用 `toArray()` 代替"。要详细学流式处理去看官方文档,速成阶段不深究。

十、`Stream` API 速查(低优先级,扫一眼即可)

范围:本节只列最常用 4-5 个 Java 8 写法。Stream 在刷题里不推荐作为主力(性能略差),但作为辅助(转换、过滤)很方便。

>

金句:"Stream 写法更函数式,但性能略差,刷题优先用 for 循环 + 工具类,Stream 用于辅助"。

10.1 4 个常用写法



```

import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6);

        // 1. filter + collect(替代手写过滤循环)
        List<Integer> evens = list.stream()
            .filter(x -> x % 2 == 0)
            .collect(Collectors.toList()); // [4, 2, 6]

        // 2. mapToInt.sum(替代 accumulate 求和)
        int sum = list.stream()
            .mapToInt(Integer::intValue) // Integer → int
            .sum(); // 30
        // C++:int sum = accumulate(list.begin(), list.end(), 0);

        // 3. distinct(替代 unique 去重)
        List<Integer> unique = list.stream()
            .distinct()
            .collect(Collectors.toList()); // [3, 1, 4, 5, 9, 2, 6]
        // C++:vector<int> unique(list.begin(), list.end());
        // unique(unique.begin(), unique.end());

        // 4. sorted(替代 sort)
        List<Integer> sorted = list.stream()
            .sorted()
            .collect(Collectors.toList()); // [1, 1, 2, 3, 4, 5, 6, 9]
        // C++:sort(list.begin(), list.end());

        // 5. 数组转 Stream(注意:基本类型数组要特殊处理)
        int[] arr = {1, 2, 3};
        int arrSum = Arrays.stream(arr).sum(); // 推荐:Arrays.stream
        int arrSum2 = IntStream.of(arr).sum(); // 等价:IntStream.of
        // 注意:Stream.of(arr) 错误!会把整个 int[] 当一个元素
    }
}

```

10.2 实战建议

场景	推荐	Stream 替代
求和 / 累加	for 循环	<code>`mapToInt.sum()`</code>
去重	<code>`HashSet`</code> 重建	<code>`stream.distinct()`</code>
排序	<code>`Arrays.sort`</code> / <code>`List.sort`</code>	<code>`stream.sorted()`</code>
过滤	for 循环 + if	<code>`stream.filter()`</code>
转 List	<code>`new ArrayList<>(collection)`</code>	<code>`collect(Collectors.toList())`</code>

金句:"Stream 用于转换和过滤,for 循环用于累计和复杂逻辑"。刷题主力还是 for 循环 + 工具类,Stream 知道有这玩意就行。



十一、API 速查总表(本章 8 大主题)

#	工具类	必会 API	重要度
1	`Arrays`	`sort` / `binarySearch` / `fill` / `toString` / `asList` / `stream` / `copyOf` / `equals`	★★★
2	`Collections`	`sort` / `reverse` / `max` / `min` / `frequency` / `swap` / `binarySearch` / `addAll` / `disjoint`	★★
3	`Math`	`max` / `min` / `abs` / `pow` / `sqrt` / `log` / `sin` / `cos` / `PI` / `E` / `random`	★★
4	`String`	`split` / `charAt` / `indexOf` / `substring` / `equals` / `hashCode` / `contains` / `startsWith` / `endsWith` / `length` / `toLowerCase` / `toUpperCase` / `trim` / `replace`	★★★
5	`StringBuilder`	`append` / `reverse` / `toString` / `deleteCharAt` / `setCharAt` / `insert` / `length` / `charAt`	★★★
6	`Integer` / `Long` / `Double`	`parseInt` / `parseLong` / `parseDouble` / `valueOf` / `toString` / `MAX_VALUE` / `MIN_VALUE` / `bitCount`	★★
7	`Comparator`	`(a,b)->a-b` / `Comparator.comparingInt` / `.thenComparingInt` / `Comparator.reverseOrder` / `Collections.reverseOrder`	★★★
8	Java 8 必备	`getOrDefault` / `computeIfAbsent` / `List.sort` / `String.join` / `String.chars` (扫一眼)	★★★

统计:本章覆盖 8 大主题、80+ 个 API。速成目标:前 7 个主题的"必会 API"全部背下来,第 8 个主题(Java 8 必备)前 4 个 API(getOrDefault / computeIfAbsent / List.sort / String.join)必背。

十二、本章坑点速查(回到 06 详讲)



#	坑点	章节	一句话
1	`Arrays.asList` 不能 add/remove	2.5	返回定长 list,要可变需外包 `new ArrayList<>()`
2	浮点比较不要直接 `==`	4.4	用 `Math.abs(a-b) < 1e-9`
3	`==` 比 `String` 是比引用	5.4	内容比较永远 `.equals()`
4	`length` / `length()` / `size()` 混用	5.5	数组 `length`、字符串 `length()`、集合 `size()`
5	`Integer ==` 在 128 外失效	7.4	永远 `.equals()` (缓存池陷阱)
6	`Math.abs(Integer.MIN_VALUE)` 是负数	7.5	溢出陷阱,用 `long`

完整 15 大坑见 06 常见坑点速查,这里只是工具类相关的 6 个。

十三、章节小结(看完你得会)

必会项	重要度	速成优先级
`Arrays.sort` 基础类型 / 对象	★★★★	必背
`Arrays.binarySearch` (先排序!)	★★	必背
`Collections.sort` / `reverse` / `max` / `min`	★★	必背
`Math.max` / `min` / `abs` / `sqrt` / `pow`	★★	必背
`String.equals` / `length` / `charAt` / `substring`	★★★★	必背
`StringBuilder.append` / `reverse` / `toString`	★★★★	必背
`Integer.parseInt` / `bitCount` / `MAX_VALUE`	★★	必背
`Comparator` lambda + `Comparator.reverseOrder`	★★★★	必背
`Comparator` 链式调用 `comparingInt().thenComparingInt`	★★	必背
`Map.getOrDefault`	★★★★	必背
`Map.computeIfAbsent`	★★★★	必背
`List.sort` (Java 8+)	★★	必背
`String.join`	★★	必背
浮点比较用 eps	★★	必背
字符串/包装类永远 `.equals`	★★★★	必背

练习建议



- Arrays 必做:LeetCode 56 合并区间(二维数组排序 + 自定义比较器)
- Collections 必做:LeetCode 347 前 K 个高频元素(用 Collections.reverseOrder 配 PriorityQueue 做大顶堆, 见 03 深讲 3)
- Math 必做:LeetCode 50 Pow(x, n)(用 Math.pow 或手写快速幂,注意浮点精度)
- String 必做:LeetCode 14 最长公共前缀(用 indexOf / substring / charAt)
- StringBuilder 必做:LeetCode 43 字符串相乘(大数乘法,用 StringBuilder 倒着累加)
- Integer.bitCount 必做:LeetCode 191 位 1 的个数(Integer.bitCount(n) 一行)
- Comparator 链式 必做:LeetCode 406 根据身高重建队列(多键排序:h 降序 + k 升序,链式调用)
- getOrDefault 必做:LeetCode 387 字符串中第一个唯一字符(统计频次 → 找 count==1)
- computeIfAbsent 必做:LeetCode 49 字母异位词分组(字符排序当 key,computeIfAbsent 一行建邻接表)
- String.join 必做:LeetCode 606 根据二叉树创建字符串(用 String 直接 += 拼括号)

下一步

打开 05_刷题模式与模板.md,学 ACM + LeetCode 两种模式的完整骨架,10 大算法 Java 模板直接抄(排序 / 二分 / 双指针 / BFS-DFS / DP / 并查集 / 单调栈 / 前缀和 / 回溯 / 贪心)。

